

### Commentaire

D'une manière générale, vous n'avez pas le droit d'utiliser de fonction préprogrammée de Python qui résoudrait directement le problème (comme la fonction `max()` pour la question 4, par exemple). Il s'agit ici d

Pour chaque question, commencez par vous demander quels sont les paramètres de la fonction demandée (et si elle en a!), et ce qu'elle doit renvoyer (et si elle doit renvoyer quelque chose!).

A l'écrit, la plupart du temps on vous imposera un nom de fonction, en vous indiquant au passage le nom des paramètres - c'est beaucoup plus simple à corriger, et cela évite des erreurs inutiles pour les candidats, livrés à eux-mêmes devant leur copie. A l'oral vous n'aurez pas forcément cette aide.

### Commentaire

Ce TP vous invite à de nombreuses reprises à consulter le cours de Python. Quand vous tombez sur une telle injonction, si vous maîtrisez déjà le paragraphe demandé, vous pouvez simplement le relire en diagonale pour vérifier et passer à la suite.

### Commentaire

**Attention !** Vous devez toujours commencer par expliquer en français votre idée avant de l'implémenter. N'oubliez pas de commenter (notamment à quoi servent les variables), utilisez des noms explicites pour vos variables, et mettez des espaces un peu partout pour rendre lisible votre prose (en particulier pour compter les parenthèses...)

## Exercice 1

Commencer par voir le *Poly de Python*, paragraphes 2.1, 2.2 et 4.3

1. Soit  $f : x \mapsto \begin{cases} 0 & \text{si } x \in [0, 1] \cup [2, 5[ \\ 1 & \text{si } x \in ]1, 2[ \\ 2 & \text{sinon} \end{cases}$ .

Faire une fonction `foncDeux(x)`, qui prend un réel  $x$  et qui renvoie  $f(x)$ .

2. a) Que fait la fonction suivante ?

```
def mafonc(n) :  
    return (n % 2) == 0
```

- b) Une fois définie, que produisent les commandes suivante :

```
>>> mafonc(3)  
>>> mafonc(2)  
>>> type(mafonc(3))
```

### Commentaire

Avec-vous compris les mots-clés `True` et `False` ? Leur type ? L'intérêt des variables de ce type et le genre d'opérations qu'on peut faire avec ?

3. Faire une fonction `foncTrois(x)`, qui prend un réel  $x$  et qui renvoie `True` si  $x \in [0, 1] \cup [2, 5[$  et `False` sinon.

4. Que produit la commande suivante :

```
>>> True + True + False + True
```

et celle-ci :

```
>>> True * True * False * True
```

## Exercice 2

1. Prendre le poly de Python, et vérifier (en faisant les exercices si nécessaire) que vous maîtrisez les paragraphes 4.4.1 et 4.4.2. L'opérateur `+` est-il commutatif pour les chaînes de caractères ?

2. Faites un programme qui demande à l'utilisateur :

— « entrez votre nom » (et qui stocke le nom dans une variable)

— « *(nom entré précédemment)*, entrez un nombre » (et qui stocke le nombre dans une variable)

et enfin qui affiche :

*(le nom de l'utilisateur), le carré de votre nombre est : (suivi de la valeur effective de  $x^2$ )*

### Commentaire

Vous aurez besoin de savoir *transtyper*. .Brrrr... Keskeusséé ?? Voir le paragraphe 4.9 du poly.

3. Prendre le poly de Python, et vérifier (en faisant les exercices si nécessaire) que vous maîtrisez les paragraphes 4.5.1, 4.5.2 et 4.5.3

4. Faire une fonction `frequence(machaine, caract)` qui renvoie la fréquence du caractère `caract` dans la chaîne `machaine`.

5. Faire une fonction `remplace_TOUS_car(chaine, caract, remplaceant)` qui renvoie une copie de la chaîne de caractères `chaine` où toute occurrence du caractère `caract` est remplacée par le caractère `remplaceant`.

6. Faire une fonction `remplace_UN_car(chaine, caract, remplaceant)` qui renvoie une copie de la chaîne de caractères `chaine` où la première occurrence du caractère `caract` est remplacée par le caractère `remplaceant`.

7. A l'aide d'une boucle, faire une fonction `es_tu_la(chaine, mot)` qui renvoie `True` si la chaîne `mot` est présente dans la chaîne de caractères `chaine`.

### Commentaire

La commande `in` permet de faire ça en une ligne (et plus vite). Si vous en avez besoin dans un programme long, vous pouvez l'utiliser, mais dans une question où c'est le BUT comme ici, il faut refaire ça à la main.

8. Faire une fonction `estunpalindrome(machaine)` qui renvoie `True` si la chaîne `machaine` est un palindrome et `False` sinon.

### Commentaire

Un palindrome est une chaîne de caractères qui peut être lue dans les deux sens sans changer.

9. (après avoir lu les paragraphes 3.8 et 4.4.4) Même question, mais la différence entre majuscules et minuscules ne doit pas compter (ainsi `kAyaK` est un palindrome alors qu'il ne l'était pas pour la question précédente).

### Exercice 3

1. Prendre le poly de Python, et vérifier (en faisant les exercices si nécessaire) que vous maîtrisez les paragraphes 4.6.1 et 4.6.2, puis le paragraphe 3.9.

2. Considérons le script suivant.

On ne demande pas de l'exécuter. La partie liée à Matplotlib n'est pas urgente.

```
import matplotlib.pyplot as plt
from math import sin exp # A quoi servent ces deux premières lignes

def fonk(x) :
    return sin(x)*exp(x)

def fonction_mystere_1(a,b,n) :
    '''
    Que fait cette fonction ?
    Que représentent ses paramètres ?
    Que renvoie-t-elle ?
    '''
    reponse = []
    p = (b-a)/n # A quoi sert cette variable ?
    for k in range(n+1) :
        reponse.append(a+k*p)
    return reponse

def fonction_mystere_2( l, f) :
    '''
    Que fait cette fonction ?
    Que représentent ses paramètres ?
    Que renvoie-t-elle ?
    '''
    reponse = []
    for element in l :
        reponse.append(f(element))
    return reponse

a , b = -1, 10
# A quoi servent a et b ? Ce 'a' est-il le même que celui de la fonction_mystere_1() ?

Nb = 1000 # A quoi sert cette variable ?

x = fonction_mystere_1(a,b,Nb)
y = fonction_mystere_2(x, fonk)

plt.figure("ceci n'est pas un graphe")
plt.plot(x,y)
plt.show()
```

a) Que fait-il ? (en une ligne)

b) Répondre aux questions posées en commentaire (ce seraient les commentaires à faire dans le code)

c) Pourquoi avoir défini à part la fonction `fonk(x)` plutôt que la mettre directement où on en a besoin ?

d) (optionnelle ! question à faire après avoir compris le paragraphe 4.6.3 du cours de Python). Redéfinir les deux fonctions précédentes à l'aide de la syntaxe « en compréhension ».

3. Considérons la suite définie par  $u_0 = 2$  et  $\forall n \in \mathbb{N}, u_{n+1} = \sqrt{1 + u_n^2}$ .

a) Faire une fonction `unTerme(n)` qui renvoie le  $n^{\circ}$  terme de la suite.

b) Faire une fonction `listeDeTermes(n)` qui renvoie la liste  $[u_0, u_1, \dots, u_n]$ .

c) En utilisant le module `matplotlib`, faire une fonction `traceTermes(n)` qui trace les termes  $[u_0, \dots, u_n]$

4. Faire une fonction `maximum(maliste)` qui renvoie le maximum de la liste `maliste`, supposée composée de nombres.

5. Faire une fonction `estendouble(maliste, objet)` qui vérifie si l'objet `objet` est présent (au moins) en double dans la liste `maliste` (renvoie `True` dans ce cas, et `False` sinon.)

6. Faire une fonction `yadesdoubles(maliste)` qui vérifie si la liste `maliste` contient au moins un élément en double (renvoie `True` dans ce cas, et `False` sinon.).

#### Exercice 4

Avant de vous faire travailler, commençons par expliquer le concept d'algorithme « en place ».

Un algorithme (cas typique un algorithme de tri de listes) est dit « en place » lorsqu'il ne renvoie pas de résultat, mais modifie la liste donnée en paramètre.

• Avant de donner des exemples, il faut bien comprendre que ceci n'a de sens que pour les paramètres de type `list`, `numpy.array` et `dict` ! En effet, ce sont les seuls types de variable *mutables* au programme en BCPST. Pour le moment, le seul type de variable de ce type qu'on va étudier sont les listes. Exemple :

```
>>> ma_chaine = "bou"
>>> ma_liste = [6,7,8]
>>> ma_chaine[0] = "Z"
```

Traceback (most recent call last):

File "<console>", line 1, in <module>

TypeError: 'str' object does not support item assignment

```
>>> ma_liste[0] = 1
>>> ma_liste
[1, 7, 8]
```

#### Commentaire

Avant les exemples, profitons-en pour rappeler la différence de syntaxe, expliquée entre autres dans les paragraphes 3.8, 4.4.4 et 4.9.3 du poly de cours, entre une *méthode* et une fonction *fonction* (ces deux termes étant à prendre ici au sens de Python).

• une fonction `ma_fonction()` s'applique comme une fonction mathématique.

Syntaxe : `ma_fonction(ses_parametres)`. Exemple : `len(ma_liste)`

• une méthode `ma_methode()` EST ATTACHEE à un objet par l'opérateur « . ».

Syntaxe : `mon_objet.ma_methode(parametres_de_la_methode)`.

Exemple : `ma_liste.append(un_objet)`

- **Passons à l'exemple des méthodes de tri fournies par Python** : la méthode `sort()` et la fonction `sorted()`.

- La méthode `sort()` : Elle implémente un algorithme de tri « en place » :

```
>>> maliste = [3,4,1,5]
```

```
>>> maliste.sort()
```

```
>>> maliste  
[1, 3, 4, 5]
```

Comme cet algorithme est « en place », cette méthode ne « renvoie » rien :

```
>>> maliste = [3,4,1,5]
```

```
>>> a = maliste.sort()
```

```
>>> print(a)  
None
```

- Par contre, la fonction `sorted()` n'est pas « en place » : elle ne modifie pas la liste donnée, mais crée et renvoie une nouvelle liste :

```
>>> maliste = [3,4,1,5]  
>>> a = sorted(maliste)  
>>> a  
[1, 3, 4, 5]  
>>> maliste  
[3, 4, 1, 5]
```

#### Commentaire

Je ne parle pas du caractère « en place » comme d'un but à atteindre mais simplement pour une raison pratique : si vous utilisez un algorithme, il faut bien savoir s'il produit une nouvelle liste ou s'il modifie celle qu'on lui donne !

Allez, à vous maintenant :

1. Avez-vous compris ce qui précède ?

2. Approfondissons les itérables : chaînes de caractères, listes, mais aussi « tuples » et « range »

Lire ou relire les paragraphes **4.5** à **4.9** du cours de Python (en faisant notamment bien les deux derniers exercices de ce dernier paragraphe).